

11 流水车间与作业车间：作业要走过好几台机器

前两章的作业只在一台机器（或一组相同的机器）上加工一次。实际的作业常常要依次经过好几道工序：托盘入库、上提升机、出库；一个订单要经过几个站点。这一章看两种最基本的多工序模型：所有作业走同样路线的流水车间，和每个作业路线不同的作业车间。前者在两台机器时有漂亮的规则，后者要用析取图来表示，并且从这里开始，建模方式（MIP 还是 CP）会明显影响能解多大的问题。

学完本章，你能

1. 用递推公式算出流水车间任意顺序的 $C_{\max}$ ，手写 Johnson 规则解 $F2 \parallel C_{\max}$ ，并说明为什么三台以上没有这样的规则。
2. 用 NEH 启发式解一般的流水车间，看出它为什么比随便排好，又为什么不保证最优。
3. 把作业车间的一个调度写成析取图：选定每台机器上的作业顺序， $C_{\max}$ 就是最长路，出现环就说明顺序不可行。
4. 为同一个作业车间问题写出大 M 的 MIP 和 CP-SAT 两种模型，并解释为什么规模一大，两者差距悬殊。

本章信息

用时	阅读与交互 80-110 分钟，Lab 3-4 小时
前置	第 9、10 章（调度记号、 $C_{\max}$ 、交换论证）；第 4 章的大 M 建模。会读 Python 代码。
对拍	SciPy 1.17.1 · HiGHS 1.12.0 · OR-Tools 9.15；页面里的算法由内置代码实现，不依赖求解器，11.4 的对比表来自 Lab 参考答案的实测。
配套	Lab 代码包：5 个练习 + 1 个挑战，含流水车间 $C_{\max}$ 与 Johnson、NEH、析取图求值、MIP 与 CP-SAT 两种模型、派工规则，pytest 自动判分

第三部分 调度理论：调度问题的语言 → 单机与并行机 → 流水车间与作业车间（本章） → 资源与时间窗约束。

11.1 流水车间与 Johnson 规则

流水车间 (flow shop) 里有 m 台机器，每个作业都按同样的次序经过它们：先第 1 台，再第 2 台……记号是 $F_m || C_{\max}$ 。作业 j 在第 k 台机器上的加工时间是 p_{jk} 。入库 → 提升机 → 出库就是一个 F3。

每台机器一次只做一个作业，一个作业同一时刻只在一台机器上。最常见的限制是**置换流水车间** (permutation flow shop)：所有机器上的作业顺序相同，于是决策只有一个，就是作业的排列。对 $C_{\max}$ 而言，两台和三台机器时最优调度总可以取成置换的，所以下面只看排列。

给定排列，各作业在各机器上的完工时间由一个递推给出。设 C_{jk} 是排在第 j 位的作业在机器 k 上的完工时间：

完工时间递推 $C_{jk} = \max(C_{j-1,k}, C_{j,k-1}) + p_{jk}$ 要等这台机器上一个作业做完，也要等自己在上一台机器上做完

$C_{\max}$ 就是最后一个作业在最后一台机器上的完工时间。这个递推是 $O(nm)$ 的，所以评价一个排列很便宜，难的是在 $n!$ 个排列里找最好的。

两台机器：Johnson 规则 (1954)。 $F_2 || C_{\max}$ 有一个多项式的最优算法。记作业 j 在两台机器上的时间为 a_j, b_j ：

- 1 把 $a_j \leq b_j$ 的作业放进集合 1，把 $a_j > b_j$ 的放进集合 2。
- 2 集合 1 按 a 从小到大排在前面，集合 2 按 b 从大到小排在后面。

直觉是两头夹：第二台机器要等第一台送来作业才能开工，所以让 a 小的作业先走，第二台机器早早有活干；最后一个作业在第二台机器上做完就结束了，所以让 b 小的作业留到最后，收尾的空转最短。证明用的是交换论证，和上一章一样：对相邻的两个作业 i, j ，若 $\min(a_i, b_j) \leq \min(a_j, b_i)$ ，则把 i 排在 j 前面不会更差。这个两两比较的关系是可以传递的，所以能推出整个顺序。

先预测 6 个作业在两台机器上的加工时间 (a, b) : J1 (1,4)、J2 (9,9)、J3 (6,5)、J4 (3,2)、J5 (5,4)、J6 (1,5)。按 J1...J6 的顺序, C_max = 35。Johnson 规则得到的顺序, C_max 是多少?

33

31

29

27

预测正确。 31。集合 1 ($a \leq b$) 是 J1、J6、J2, 按 a 升序; 集合 2 是 J3、J5、J4, 按 b 降序, 得到 J1→J6→J2→J3→J5→J4。它等于枚举 720 种顺序的最小值。C_max 至少是 $\max(\sum a + \min b, \min a + \sum b) = 30$, 所以 29 和 27 不可能。下面的图已经解锁。



图 11-1 两台机器的流水车间（M1 是入库口，M2 是提升机）。方块里是作业编号，红色虚线是 $C_{\max}$ 。用 ◀▶ 可以手动调整任何一个位置，也可以直接套用预置的顺序。Johnson 顺序总是最优；最优顺序不止一个（这个例子有 12 个），Johnson 只是其中一个。

读图

原顺序 J1...J6 : $C_{\max} = 35$ 。J2 在第一台机器上要 9，第二台机器在这段时间里几乎全部空着。

Johnson 顺序 : 集合 1 是 J1、J6、J2 (a 依次为 1、1、9，并列取下标小的)，集合 2 是 J3、J5、J4 (b 依次为 5、4、2)。得到 $J1 \rightarrow J6 \rightarrow J2 \rightarrow J3 \rightarrow J5 \rightarrow J4$ ， $C_{\max} = 31$ ，与枚举 720 种得到的最优一致。**最差顺序** : 41，比最优多 10。

试着把 Johnson 顺序里的两个作业交换。绝大多数交换会让 $C_{\max}$ 变大或不变，不会变小。这就是最优的含义。

三台机器

F3 || $C_{\max}$ 是强 **NP 难** 的 (Garey、Johnson、Sethi, 1976)，所以不能指望像 Johnson 那样的规则。有一个特殊情形可以化简：若第二台机器被“压住”，即 $\min a \geq \max b$ 或 $\min c \geq \max b$ (a 、 b 、 c 是三台机器的时间)，那么把每个作业合成 $(a+b, b+c)$ 两台虚拟机器，再用 Johnson 规则就是最优的。一般情形没有这个性质，下一节用启发式。

11.2 三台以上：NEH 启发式

机器多了以后，实际中最常用的构造启发式是 **NEH** (Nawaz、Enscore、Ham, 1983)。它的想法很简单：加工时间大的作业更“难安排”，先安排它们；每个作业都放在当前最合适的位置。

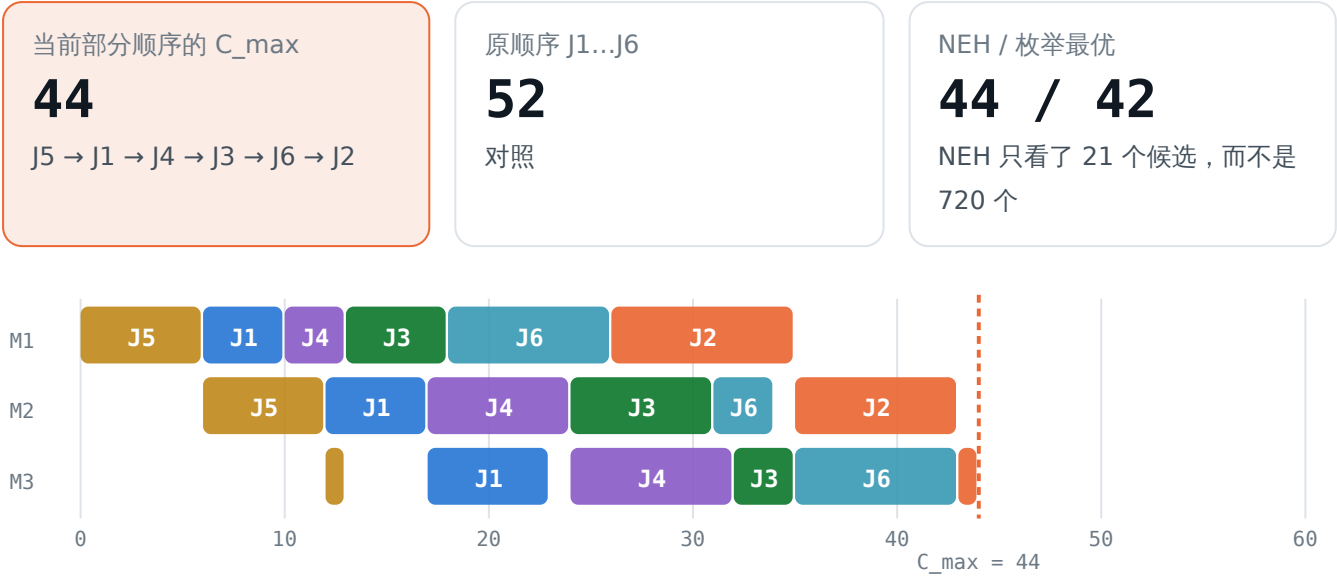
- 1 按总加工时间 $\sum_k p_{jk}$ 从大到小排出插入次序。
- 2 取第一个作业，形成只有一个作业的部分顺序。
- 3 依次取下一个作业，试着把它插进部分顺序的每一个位置（含最前和最后），用完工时间递推算出每个位置的 $C_{\max}$ ，保留最小的（并列取靠前的位置）。

插入第 k 个作业要试 k 个位置，总共 $O(n^2)$ 次评价， $n = 100$ 也只需要几毫秒。它在流水车间的文献里几乎是“必比的基线”，很多更复杂的方法（禁忌搜索、迭代贪心）都以 NEH 的解为起点。

先预测 3 台机器、6 个作业（图 11-2 的数据），按 J1...J6 的顺序 $C_{\max} = 52$ 。NEH 只评价 21 个候选，得到的 $C_{\max}$ 是多少？（枚举 720 种的最优值在选项之中。）

- 46
- 44
- 42
- 40

预测正确。 44。NEH 比原顺序好 8，与最优 42 差 2。选项里 42 是最优值，NEH 没有保证能到；40 比最优还小，不可能。下面的图已经解锁。



第 6 步：插入 **J5**。可选位置 6 个， $C_{\max}$ 依次是 44、44、44、44、44、44，选 **第 1 个位置** (44)。点表里的一行可以预览别的位置。

结束。 NEH 得到 J5 → J1 → J4 → J3 → J6 → J2， $C_{\max} = 44$ ；原顺序 52；枚举 720 种的最优是 42，NEH 差 2。

位置	部分顺序	$C_{\max}$
1	J5 J1 J4 J3 J6 J2	44
2	J1 J5 J4 J3 J6 J2	44
3	J1 J4 J5 J3 J6 J2	44
4	J1 J4 J3 J5 J6 J2	44
5	J1 J4 J3 J6 J5 J2	44
6	J1 J4 J3 J6 J2 J5	44

图 11-2 NEH 在 3 台机器、6 个作业（入库口 M1、提升机 M2、出库口 M3）上的逐步插入。表里每一行是一个候选位置，加粗的行是被选中的位置；点其他行可以预览它的甘特图。

读图

总加工时间是 J1 15、J2 18、J3 15、J4 18、J5 13、J6 19，所以插入次序是 J6、J2、J4、J1、J3、J5（并列取下标小的）。部分顺序的 $C_{\max}$ 依次是 19、26、29、33、38、44。最后 NEH 得到 J5 → J1 → J4 → J3 → J6 → J2， $C_{\max} = 44$ ；原顺序 J1...J6 是 52；枚举 720 种得到的最优是 42。NEH 一共只评价了 $1 + 2 + 3 + 4 + 5 + 6 = 21$ 个候选，把 52 压到 44，与最优差 2。

它不保证最优。第 5 步插入 J3 时五个候选是 41、40、38、38、42，最小值 38 出现两次，规则取靠前的那个。换一个并列规则，后面的结果可能不同，所以 Lab 的练习 2 把并列规则写死，让结果可以复现。

11.3 作业车间与析取图

作业车间 (job shop) 里，每个作业有自己的路线：作业 J1 先上 M3，再 M2，最后 M1；作业 J2 的路线是 M3、M1、M2。同一台机器一次只做一个作业。记号 $J^m || C_{\max}$ 。四向车仓库里“每个托盘要经过不同的一串站点”就是这个模型。它比流水车间难得多：流水车间是它的特例，所以 $J3 || C_{\max}$ 也是强 NP 难的； 10×10 的经典实例 (Fisher-Thompson, 1963) 在提出后二十多年里都没有被证明最优。

用析取图 (disjunctive graph) 来表示：

- 节点是工序（作业 j 的第 k 道），节点权重是它的加工时间。
- 路线弧（实线灰色）：同一作业的相邻工序，前一道做完才能做后一道。
- 析取弧：同一台机器上的两个工序不能同时做，要么 A 在 B 前，要么 B 在 A 前，这是一个二选一（“析取”）。选定每台机器上的作业顺序，就是为每一对二选一定下方向，得到一张有向图。

一个选择对应一个可行调度，当且仅当这张有向图没有环。有环意味着“A 要等 B，B 要等 C，C 又要等 A”，谁也无法开工。无环时，每道工序的最早开始时刻就是从起点到它的最长路， $C_{\max}$ 是整张图里的最长路，最长路上的工序叫**关键工序**，缩短不在关键路径上的工序不会让 $C_{\max}$ 变小。

析取图求值 $C_{\max}$ = 图中的最长路（按拓扑序一遍扫描），有环则不可行 $O(\text{工序数} + \text{弧数})$

所以作业车间的搜索空间是“每台机器上的一个排列”，共 $(n!)^m$ 个组合，其中一部分带环。

先预测 3 个作业、3 台机器的作业车间。每台机器上的作业顺序各有 $3! = 6$ 种，共 $6 \times 6 \times 6 = 216$ 种组合。其中能对应可行调度（析取图无环）的有多少种？

216

150

63

27

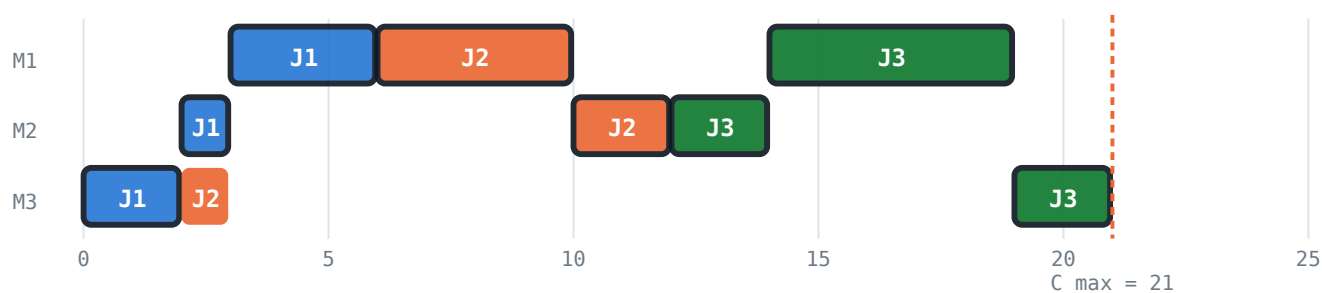
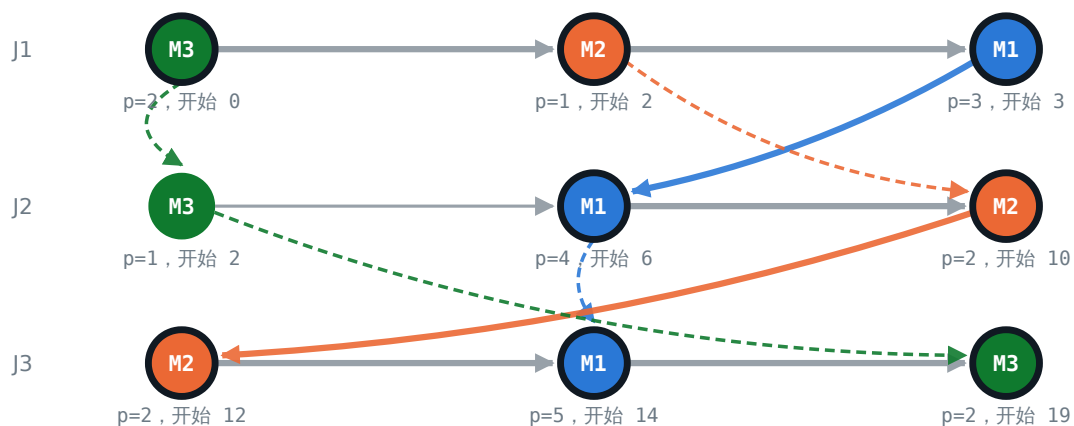
预测正确。 63。不到三分之一的组合是可行的，其余都会让某几个作业互相等待，析取图里出现环。这说明直接在机器顺序上搜索时，必须处理不可行的邻居。下面的图已经解锁。

预置 每台机器 J1→J2→J3 最优 有环的例子



可行 (析取图无环)

比最优 13 多 8



$C_{\max}$ (最长路)

21

黑框的工序是关键路径

下界 (最忙的机器或最长的作业)

12

最优值 13 比它大 1

可行的机器顺序

63 / 216

每台机器 $3! = 6$ 种, 共 $6^3 = 216$ 种

关键路径 (最长路) : J1·M3 → J1·M2 → J1·M1 → J2·M1 → J2·M2 → J3·M2 → J3·M1 → J3·M3, 长度 **21**。缩短不在关键路径上的工序不会改变 $C_{\max}$ 。

图 11-3 3 个作业、3 台机器 : J1 走 M3→M2→M1 (时间 2、1、3) , J2 走 M3→M1→M2 (1、4、2) , J3 走 M2→M1→M3 (2、5、2) 。节点颜色是机器, 灰色实线是路线弧, 彩色虚线弧是选定的机器顺序; 黑框和粗实线是关键路径。用每台机器上的 ◀▶ 改变顺序, 图和甘特图同步变化。

读图

每台机器都按 **J1、J2、J3**：可行， $C_{\max} = 21$ ，关键路径 $J1 \cdot M3 \rightarrow J1 \cdot M2 \rightarrow J1 \cdot M1 \rightarrow J2 \cdot M1 \rightarrow J2 \cdot M2 \rightarrow J3 \cdot M2 \rightarrow J3 \cdot M1 \rightarrow J3 \cdot M3$ ，几乎把所有工序串成了一条线。**最优**： $C_{\max} = 13$ ，三台机器上的顺序分别是 $M1 : J2、J3、J1$ ， $M2 : J3、J2、J1$ ， $M3 : J2、J1、J3$ ；关键路径是 $J2 \cdot M3 \rightarrow J2 \cdot M1 \rightarrow J3 \cdot M1 \rightarrow J1 \cdot M1$ ，长度 $1 + 4 + 5 + 3 = 13$ 。下界（最忙的机器或最长的作业）是 12，最优比它大 1。

有环的例子： $M1、M2$ 上都是 $J1、J2、J3$ ， $M3$ 上却是 $J3、J2、J1$ 。图里出现红色的环，页面给出具体的环。**216 种机器顺序里只有 63 种是可行的**，不到三分之一。这个比例在大规模问题里会更小，所以直接在“机器顺序”上做局部搜索时，要小心邻域产生的解是否有环。

需要留意，一个可行的机器顺序对应“半主动调度”：每道工序都尽可能早开始。 $C_{\max}$ 是正规目标（完工时间变晚不会变好），所以最优调度总可以取成半主动的，搜索机器顺序就够了。

11.4 两种建模：大 M 的 MIP 与 CP-SAT

同一个作业车间问题，可以写成混合整数规划，也可以写成约束规划（CP）。它们描述的是同一个问题，但求解器的表现差别很大。这一节把两个模型并排放，看差别在哪里。

大 M 的 MIP 模型

变量：每道工序的开始时刻 s_o ，最大完工时间 C ，以及同一台机器上每一对工序 (a, b) 的顺序变量 $y_{ab} \in \{0, 1\}$ （1 表示 a 在 b 之前）。约束：

路线 $s_{j,k+1} \geq s_{jk} + p_{jk}$ 同一作业的相邻工序

析取 $s_b \geq s_a + p_a - M(1 - y_{ab})$ ， $s_a \geq s_b + p_b - M y_{ab}$ $y = 1$ 时第一条起作用， $y = 0$ 时第二条起作用；另一条被 M 放松

目标 $\min C$ ， $C \geq s_{j,\text{末}} + p_{j,\text{末}}$ 每个作业的最后道工序

这是第 4 章讲过的“大 M”写法。它的问题也是第 4 章的问题： M 取工序时间之和（一个安全的上界），把 y 放松成 $[0, 1]$ 以后，两条析取约束几乎不起作用，LP 松弛值很小。对本章的 3×3 例子，LP 松弛值只有 **9.0**，而下界是 12、最优是 13。松弛这么弱，分支定界就得枚举大量的 y 组合才能证明最优。

CP-SAT 的模型

CP-SAT 直接支持**区间变量**和**不重叠约束**：每道工序是一个区间（开始、时长、结束），每台机器上的区间用一个 `NoOverlap` 约束，作业内部用 `start ≥ 前一道的 end`。没有 M ，没有 0/1 的顺序变量，求解器内部知道“不重叠”的含义，会做**边界推理**：比如推断“这几个工序合起来至少占用多长的时间窗”，并把传播的结果学成子句。

实例	工序数	最优 C_max	CP-SAT 用时	MIP 结果（解 / 对偶界）	MIP 用时	MIP 是否证明最优	LP 松弛
页面例子 3×3	9	13	0.20 秒	13 / 13.00	0.02 秒	是	9.0
5×5	25	28	0.01 秒	28 / 28.00	0.21 秒	是	—
8×8	64	38	0.02 秒	38 / 38.00	7.91 秒	是	—
10×10	100	47	0.06 秒	52 / 42.00	30.02 秒	否（限时 30 秒）	—
12×10	120	48	0.07 秒	57 / 41.57	30.04 秒	否（限时 30 秒）	—

表 11-1 同一批随机作业车间实例（作业数 $\times$ 机器数，每个作业走遍所有机器，加工时间 1-9）上，用 Lab 参考答案实测。MIP 是 SciPy 的 HiGHS，限时 30 秒；CP-SAT 是 OR-Tools，单线程。“MIP 结果”一栏是限时内找到的最好解 / 证明的对偶界。用时依赖机器，只有数量级有参考意义。

读表

3×3 时两者都是毫秒级，看不出差别。到 8×8 （64 道工序），MIP 花了 7.9 秒才证明最优，CP-SAT 只要 0.02 秒。到 10×10 和 12×10 ，MIP 在 30 秒内没有证明最优： 10×10 找到了 52，对偶界只到 42，最优值是 47，还有很大的间隙； 12×10 找到 57，对偶界 41.57，最优 48。CP-SAT 在同样的实例上都在 0.1 秒内证明了最优。

这个差别来自问题的结构，与 CP-SAT 是否普遍更强无关。作业车间由“不重叠”这种时间窗推理主导，CP 的模型和推理正好对口，MIP 的松弛却被大 M 破坏。反过来，如果问题的主要难点在成本的线性结构上（第 5-7 章的指派、网络流、背包），MIP 的强松弛和切平面才是优势。**选择建模语言要看问题的结构**，这是本章最重要的一条建议。

最后一个提醒，模型正确不等于模型好用。两个模型的最优值必须相同（Lab 的练习 4 用 pytest 检查这一点），求解速度才是要比的东西。

参考实现 (Lab 练习 4)

下面两段代码来自 Lab 的参考答案。第一段构造大 M 模型并交给 HiGHS；第二段是 CP-SAT。

```
1 def mip_jobshop(jobs):
2     """大 M 的析取模型，返回最优的最大完工时间（整数）。"""
3     c, A, lo, hi, n, npair, M = _mip_parts(jobs)
4     integrality = np.zeros(len(c))
5     integrality[n + 1:] = 1
6     ub = np.concatenate([np.full(n + 1, M), np.ones(npair)])
7     res = milp(c, constraints=LinearConstraint(A, lo, hi), integrality=integrality,
8     return int(round(res.fun))
```

```
1 def cp_jobshop(jobs):
2     """CP-SAT：区间变量 + 每台机器的 NoOverlap，返回最优的最大完工时间。"""
3     from ortools.sat.python import cp_model
4     model = cp_model.CpModel()
5     horizon = sum(d for job in jobs for _, d in job)
6     by_machine = {}
7     ends = []
8     for j, job in enumerate(jobs):
9         prev_end = None
10        for k, (m, d) in enumerate(job):
11            s = model.NewIntVar(0, horizon, f"s{j}_{k}")
12            e = model.NewIntVar(0, horizon, f"e{j}_{k}")
13            iv = model.NewIntervalVar(s, d, e, f"i{j}_{k}")
14            by_machine.setdefault(m, []).append(iv)
15            if prev_end is not None:
16                model.Add(s >= prev_end)
17            prev_end = e
18        ends.append(prev_end)
19    for ivs in by_machine.values():
20        model.AddNoOverlap(ivs)
21    cmax = model.NewIntVar(0, horizon, "cmax")
22    model.AddMaxEquality(cmax, ends)
23    model.Minimize(cmax)
24    solver = cp_model.CpSolver()
25    solver.parameters.num_workers = 1
26    solver.parameters.random_seed = 1
27    status = solver.Solve(model)
28    assert status == cp_model.OPTIMAL
29    return int(solver.ObjectiveValue())
```

11.5 在项目里的位置

这一章的模型和四向穿梭车仓库的对应关系很直接，但直接套用之前要先想清楚哪些假设不成立。

对应关系

- **入库口 → 提升机 → 出库口**：一批托盘依次经过三个设备，设备一次只处理一个托盘，正好是 $F3 || C_{\max}$ 。作业顺序由“先处理哪个托盘”决定，NEH 可以直接当作离线排序的基线。
- **托盘走不同的路线**：有的托盘要过提升机换层，有的不用；有的要经过分拣站，有的不用。这就是作业车间，路线不同，机器（设备）可以被多个作业竞争。
- **小规模精确，大规模换方法**：几十道工序以内，CP-SAT 可以证明最优，也可以做在线滚动求解；工序数上千以后，需要第 5、6 部分的分解和启发式。

要小心的假设

- **缓冲区是有限的**：经典流水车间假设两台机器之间有无限缓冲，作业做完后可以等着。真实的提升机前只能停几个托盘，满了上一站就被阻塞（blocking）。有限缓冲的流水车间没有 Johnson 规则，要另外建模。
- **没有转换时间**：换层、换向的时间与前后的作业有关（序相关的准备时间），第 9 章的 s_{jk} 就是这个。带上它，即使一台机器也是 TSP 型的难问题。
- **作业一开始就全部就位**： $C_{\max}$ 的排序适合“一个波次的所有托盘已知”，托盘陆续到达时要用滚动时域，或者改用 11.4 的 CP 模型每隔一段时间重新求解。

Lab 的挑战是一个最常用的**派工规则**（dispatching rule）：每次在所有“可以开始”的工序里，选最早能开始的一道，并列时选加工时间短的。它算得很快，但在本章的 3×3 例子上得到 15，最优是 13。规则给出的是可行解，而不是最优解，把它当作 CP-SAT 的初始解或者对照基线是合适的。

11.6 小结与 Lab

- 1 流水车间：所有作业走同样的机器次序，置换调度的 $C_{\max}$ 由递推 $C_{jk} = \max(C_{j-1,k}, C_{j,k-1}) + p_{jk}$ 在 $O(nm)$ 内算出。
- 2 F2 || $C_{\max}$: Johnson 规则， $a \leq b$ 的按 a 升序放前面， $a > b$ 的按 b 降序放后面，最优。第 3 台机器加入后就是强 NP 难。
- 3 NEH：按总加工时间降序，逐个插到使部分顺序 $C_{\max}$ 最小的位置。只评价 $O(n^2)$ 个候选，通常很接近最优，但没有最坏保证。
- 4 作业车间：析取图上，节点是工序，路线弧固定，选定每台机器的作业顺序即为析取弧定向。 $C_{\max}$ 是最长路，有环即不可行。 3×3 的例子，216 种机器顺序只有 63 种可行。
- 5 同一个问题的两种模型：大 M 的 MIP 松弛很弱（本章例子 LP 值 9.0，最优 13），规模一大就证明不了最优；CP-SAT 用区间变量和 NoOverlap， 10×10 、 12×10 都能在 0.1 秒内证明最优。
- 6 选建模语言要看结构：时间窗和“不重叠”的问题适合 CP，成本线性、网络结构强的问题适合 MIP。
- 7 在项目里，入库 → 提升机 → 出库是 F3，多路线的托盘是作业车间；有限缓冲、准备时间和陆续到达都会改变结论，需要另外建模。

术语表

中文	English	在本章的含义
流水车间、置换流水车间	flow shop, permutation flow shop	所有作业按同样的机器次序加工；所有机器上的作业顺序也相同
Johnson 规则	Johnson's rule	$F2 C_{\max}$ 的最优规则
NEH 启发式	NEH heuristic	按总时间降序，逐个插入最好的位置
作业车间	job shop	每个作业有自己的机器路线
析取图	disjunctive graph	工序为点，路线弧固定，同一机器上的工序两两析取
关键路径	critical path	析取图里的最长路，长度就是 $C_{\max}$
半主动调度	semi-active schedule	每道工序都不能在不改变机器顺序的前提下更早开始
区间变量、不重叠约束	interval variable, NoOverlap	CP-SAT 表示工序和机器互斥的原语
派工规则	dispatching rule	每次在可开始的工序里按优先规则选一道

对照教材

- Pinedo , *Scheduling: Theory, Algorithms, and Systems* (第 5 版 , Springer 2016) : 第 6 章流水车间 (Johnson、NEH、有限缓冲)、第 7 章作业车间 (析取图、移动瓶颈)。
- Johnson , “Optimal two- and three-stage production schedules with setup times included” (Naval Research Logistics Quarterly , 1954) 。 Garey、Johnson、Sethi , “The complexity of flowshop and jobshop scheduling” (Mathematics of Operations Research , 1976) 。
- Nawaz、Enscore、Ham , “A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem” (Omega , 1983) 。
- Roy & Sussmann , “Les problèmes d'ordonnancement avec contraintes disjonctives” (1964) : 析取图。Fisher & Thompson (1963) 的 10×10 实例。
- Google OR-Tools 文档的 Job Shop 示例 (CP-SAT 的区间变量与 NoOverlap) ; Brucker , *Scheduling Algorithms* (Springer 2007) 的作业车间章节。

本章 Lab

代码包 `ch11-shop-scheduling-lab.zip` 随本章一起发放。依赖是 `numpy`、`scipy`、`ortools`、`pytest`；判分用暴力枚举和第二个独立的求解，不依赖你的代码。

练习	内容	对应
ex1	流水车间：完工时间递推、Johnson 规则；先填 4 个预测	11.1
ex2	NEH：逐个插入，并列取靠前的位置	11.2
ex3	析取图：给定机器顺序，返回 $C_{\max}$ ，有环返回 None	11.3
ex4	两种模型：大 M 的 MIP、LP 松弛值、CP-SAT，比较最优值	11.4
ex5	审 5 个 AI 写的流水车间函数：不看源码，只靠运行结果归类问题	11.1
挑战	派工规则：最早能开始的工序优先，并列取时间短的，再取下标小的	11.5

```
pip install -r requirements.txt
pytest -q # 全部练习
pytest tests/test_ex3.py -q # 单个练习

# 确认判分本身没问题：参考答案应当全部通过
LAB_TARGET=solutions pytest -q
```

练习 1 要求先在 `PREDICTION` 里写下预测再运行；跑完测试后，终端最后会把你的预测和实际结果并排列出来。

下一章

CHAPTER 12

资源与时间窗约束

前面的工序只需要一台机器。真实的调度里，一道工序常常同时需要几种资源（提升机加一个操作员，一辆穿梭车加一条巷道），而且有开始不早于、完成不晚于的时间窗。下一章把这些写成资源受限的项目调度问题（RCPSP），看时间窗、优先关系和多资源怎样进入 CP-SAT 模型。

11.7 自测

8 道题，覆盖本章的全部要点。每题选完立即给出解析。

Q1 置换流水车间里，排在第 j 位的作业在机器 k 上的完工时间 C_{jk} 怎么算？

$$C_{jk} = C_{j,k-1} + p_{jk}$$

$$C_{jk} = \max(C_{j-1,k}, C_{j,k-1}) + p_{jk}$$

$$C_{jk} = C_{j-1,k} + p_{jk}$$

$$C_{jk} = \min(C_{j-1,k}, C_{j,k-1}) + p_{jk}$$

正确。 要等这台机器上一个作业做完 ($C_{j-1,k}$)，也要等自己在上一台机器上做完 ($C_{j,k-1}$)，取较晚的一个再加上自己的加工时间。

Q2 Johnson 规则里， $a > b$ 的作业为什么放在后面并按 b 从大到小排？

因为它们的 a 大

最后一个作业在第二台机器上的时间决定收尾的空转， b 小的留到最后，让收尾最短

因为这样第一台机器更忙

没有理由，是约定

正确。 前半部分让 a 小的先走，第二台机器早早有活干；后半部分让 b 小的留到最后，第二台机器收尾的等待最短。

Q3 F3 || C_max 为什么没有像 Johnson 那样的多项式规则？

因为三台机器没法排

它是强 NP 难的（除非 $P = NP$ ）

因为置换调度不再最优

因为递推公式失效

正确。 F3 || C_max 是强 NP 难的（Garey、Johnson、Sethi，1976）。只有 $\min a \geq \max b$ 或 $\min c \geq \max b$ 这类特殊情形，才能合成两台虚拟机器再用 Johnson。

Q4 NEH 启发式的核心步骤是？

按加工时间从小到大排序

按总加工时间降序取作业，逐个插入使部分顺序 C_max 最小的位置

随机交换两个作业直到不能改进

先解 LP 松弛再取整

正确。 NEH 只评价 $O(n^2)$ 个候选，结果通常接近最优，但没有最坏保证。

Q5 作业车间的析取图里，选定每台机器上的作业顺序以后，C_max 是什么？

所有工序的加工时间之和

最忙的那台机器的负荷

图里的最长路

最长作业的总时间

正确。 路线弧和机器弧合起来是一张有向图，每道工序最早开始时刻是起点到它的最长路，C_max 是整张图的最长路。最忙的机器负荷和最长作业只是下界。

Q6 析取图里出现环意味着什么？

C_max 很大

这组机器顺序互相矛盾，没有对应的可行调度

有多个最优解

需要更多机器

正确。 环表示“A 要等 B，B 要等 C，C 又要等 A”，谁也无法开始。本章的 3×3 例子里，216 种机器顺序只有 63 种无环。

Q7 大 M 的 MIP 模型对作业车间为什么弱？

因为变量太少

M 取得很大，析取约束在 y 放松成小数后几乎不起作用，LP 松弛值远小于最优值（例子里 9.0 对 13）

因为整数变量不能取 0 和 1

因为目标函数是非线性的

正确。 松弛太弱，分支定界要枚举很多 y 的组合才能证明最优。CP-SAT 用区间变量和 NoOverlap，不需要 M，内部有针对“不重叠”的边界推理。

Q8 关于“选 MIP 还是 CP”，本章的建议是？

永远选 CP-SAT

永远选 MIP

看问题结构：时间窗和不重叠为主的调度问题适合 CP，成本线性、网络结构强的问题适合 MIP

两者一样，随便选

正确。 同一个作业车间， 10×10 上 CP-SAT 0.1 秒内证明最优，大 M 的 MIP 30 秒还有很大的间隙；第 5-7 章的指派、网络流、背包则是 MIP 的强项。

8 / 8 本章掌握得不错，可以去做 Lab 了。

数据说明：本章的全部数字由页面内置的流水车间递推、Johnson 规则、NEH、析取图最长路与枚举实时计算，并与暴力枚举、Python 实现逐项对拍（环境：SciPy 1.17.1 · HiGHS 1.12.0 · OR-Tools 9.15）。11.4 的对比表来自 Lab 参考答案在随机实例上的实测，用时依赖机器。